# Note 3.3 Complexity of Algorithms

# Introduction

In this section, we will learn the computational complexity of the algorithm, including time complexity and space complexity.

# Time Complexity

## Definition

The time complexity is the analysis of the time required to solve a problem of a particular size. It can be expressed in terms of the number of operations used by the algorithm. The operations used can be the comparison of integers, the addition of integers, the multiplication of integers, the division of integers, or any other basic operations.

To analyze the time complexity we should find the maximum of a finite set of integers.

## Worst-Case Complexity

The type of complexity analysis done when considering the worst case is a worst-case analysis. And we need to use the largest number of operations to solve the given problem.

For example, the worst-case time complexity of the linear search is $\Theta(n)$ and binary search is $\Theta(\log n)$.

## Average-Case Complexity

The type of complexity analysis done when considering the average case is a average-case analysis. And we need to use the average number of operations to solve the given problem. Usually it is much more complicated than worst-case analysis.

## Worst-Case Complexity of Two Sorting Algorithms

By proving, both the bubble sort and the insertion sort have worst-case time complexity $\Theta(n^2)$.

# Complexity of Matrix Multiplication

Here is the algorithm of matrix multiplication.

**ALGORITHM 1 Matrix Multiplication.**

**procedure** *matrix multiplication*(**A**, **B**: matrices)
**for** $i := 1$ **to** $m$
  **for** $j := 1$ **to** $n$
    $c_{ij} := 0$
    **for** $q := 1$ **to** $k$
      $c_{ij} := c_{ij} + a_{iq}b_{qj}$
**return** **C** {$\mathbf{C} = [c_{ij}]$ is the product of **A** and **B**}

Suppose $A, B$ are all $n \times n$ matrix, then every element in $C$ need $n$ multiplication and $(n-1)$ addition, thus a total $n^3$ operators are needed. Thus the complexity of matrix multiplication is $O(n^3)$ by the definition. (Other algorithms can have $O(n^{\sqrt{7}})$). If $A$ is $m \times n$ matrix and $B$ is $n \times r$ matrix then the complexity is $O(mnr)$.

And The Boolean Product in chapter 2 Boolean Product has the algorithm as following:

**ALGORITHM 2 The Boolean Product of Zero-One Matrices.**

**procedure** *Boolean product of Zero-One Matrices* (**A**, **B**: zero–one matrices)
**for** $i := 1$ **to** $m$
  **for** $j := 1$ **to** $n$
    $c_{ij} := 0$
    **for** $q := 1$ **to** $k$
      $c_{ij} := c_{ij} \vee (a_{iq} \wedge b_{qj})$
**return** **C** {$\mathbf{C} = [c_{ij}]$ is the Boolean product of **A** and **B**}

It is similar to the matrix multiplication thus it is also has the complexity $O(n^3)$.

## Matrix-Chain Multiplication

The method to calculate the matrix-chain $A_1A_2\cdots A_n$ where $A_1, A_2, \cdots, A_n$ are $m_1 \times m_2$, $m_2 \times m_3$, $\cdots$, $m_n \times m_{n+1}$ we need to first calculate the minimum $m_k \times m_{k+1}$.

# Algorithmic Paradigms

## Definition

Algorithmic paradigm is a general approach based on a particular concept that can be used to construct algorithms for solving a variety of problems.

## Brute-Force Algorithms

For a brute-force algorithms, a problem is solved in the most straightforward manner based on the statement of the problem and the definitions of terms.

For example, to use the brute-force algorithm to find the closest pair of points,

**ALGORITHM 3 Brute-Force Algorithm for Closest Pair of Points.**

**procedure** *closest-pair*$((x_1, y_1), (x_2, y_2), \ldots, (x_n, y_n)$: pairs of real numbers)
$min = \infty$
**for** $i := 2$ **to** $n$
    **for** $j := 1$ **to** $i - 1$
        **if** $(x_j - x_i)^2 + (y_j - y_i)^2 < min$ **then**
            $min := (x_j - x_i)^2 + (y_j - y_i)^2$
            $closest\ pair := ((x_i, y_i), (x_j, y_j))$
**return** *closest pair*

And the complexity is $O(n^2)$.

# Understanding the Complexity of Algorithms

| Complexity | Terminology |
|---|---|
| $\Theta(1)$ | constant complexity |
| $\Theta(\log n)$ | logarithmic complexity |
| $\Theta(n)$ | linear complexity |
| $\Theta(n \log n)$ | linearithmic complexity |
| $\Theta(n^b)$ | polynomial complexity |
| $\Theta(b^n)$ where $b > 1$ | exponential complexity |
| $\Theta(n!)$ | factorial complexity |

## Tractability

A problem that is solvable using an algorithm with polynomial worst-case complexity is called tractable.
Problems that cannot be solved using an algorithm with worst-case polynomial time complexity. Such problems are called intractable.
Some problems even exist for which it can be shown that no algorithm exists for solving them. Such problems are called unsolvable.

## P Versus NP

Problems for which a solution can be checked in polynomial time are said to belong to the class NP (tractable problems are said to belong to class P)
The NP-complete problems is the problems with the property that is if any of these problems

can be solved by a polynomial worst-case time algorithm, then all problems in the class NP can be solved by polynomial worst-case time algorithms.

The P versus NP problem asks whether NP, the class of problems for which it is possible to check solutions in polynomial time, equals P, the class of tractable problems.